

# GOLD-001 — Batch 005

## Accelerated Coverage, and Why It Returned 19

Production RAG v1 · evidence-candidate authoring · generated 2026-08-23T16:29:55Z · nothing verified, nothing gold

### THE HEADLINE IS THE SHORTFALL

The target was **30**. This batch returns **19**. The first draft did produce 30, and reading them found the same defect again and again: the question's subject was not the fact's subject. Those defects were fixed in the miners rather than filtered at the end, and the candidates that depended on them are gone.

Nothing here is verified. Every candidate is `candidate_unverified`, and the project's confirmed total is still **67** holdout-eligible cases from batches 001–004.

|                                                                      |                                                                                         |                                                                                             |                                                                         |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <div>19 / 30</div> <div>candidates returned<br/>against target</div> | <div>27 / 46</div> <div>dropped by the self-review<br/>after passing the precheck</div> | <div>3 vs 559</div> <div>multi-hop pairs considered<br/>dependency-first vs batch 004</div> | <div>67</div> <div>confirmed eligible<br/>unchanged by this batch</div> |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|

### 1. Starting state

Read from `experiments/GOLD-001/GOLD-001-eligibility-status.json` before generating anything: **67 human\_verified**, **67 holdout\_eligible**, 4 rejected, 1 genuine multi-hop. Holdout frozen: false.

### 2. Why nineteen

The first draft produced thirty. Reading every one of them surfaced one dominant defect class and several smaller ones, all of which were fixed in the miners so the bad candidates were never built:

- The question's subject was not the fact's subject.** "What is the documented limit on `request_too_large`?" — the limit is on the request; the error code is what you get for exceeding it. The miners now take the identifier that governs the matched verb and drop the candidate when there is none.
- Splits through a literal.** A conditional split inside a JSON object produced a question ending mid-brace.
- Outcomes with no verb.** "...returns `stop_reason: 'refusal'`", not an error" was split so that "not an error" became the answer.
- Wishes as conditions.** "What happens if you want to change this?" — a reader's intention is not a state of the system, and "this" names nothing in the span.
- Table rows, stranded list ordinals, and literal subjects** — "What does `None` turn off?" asks a question about a keyword.
- Relations with the direction reversed.** "What does `betas` override?" where the source says the model *rejects* caller-supplied `betas` overrides.

### THE LARGEST SINGLE CUT WAS DELIBERATE

Bare definition bullets — "What is the `insert_line` option?" — take their scope from the heading above them. Batch 004's review rejected three candidates of exactly that shape, because §19 forbids leaning on a heading outside the span. Shipping them again would have been a knowing regression, so the self-review drops them: five candidates, and the reason is recorded rather than the count preserved.

### 3. What the batch contains

| reasoning type                                 | in batch | target | met    | eligible available |
|------------------------------------------------|----------|--------|--------|--------------------|
| <code>genuine_multi_hop</code>                 | 0        | 0–6    | met    | 0                  |
| <code>error_behavior</code>                    | 3        | 5–6    | MISSED | 3                  |
| <code>configuration_interaction</code>         | 9        | 5–6    | MISSED | 9                  |
| <code>exact_lookup</code>                      | 2        | 5–6    | MISSED | 2                  |
| <code>lifecycle_compatibility_migration</code> | 4        | 3–4    | met    | 4                  |
| <code>ambiguity_disambiguation</code>          | 1        | 3–4    | MISSED | 1                  |
| <code>comparison</code>                        | 0        | 2–3    | MISSED | 0                  |

Provider: {'anthropic': 8, 'openai': 11} across 16 distinct documents ({'anthropic': 7, 'openai': 9}). Both provider targets are missed, and the reason is the same as the category shortfalls: the pool that survived the checks was smaller than the mixture needed.

#### OVERFLOW, DECLARED RATHER THAN QUIET

**3 of 19** candidates were taken beyond §7's ceilings. Ambiguity, comparison and multi-hop are corpus-limited here, so holding every ceiling would have returned a batch far shorter still while vetted candidates waited in the categories that do have material. Rather than raise a ceiling silently, the shortfall is filled under a declared cap and each such candidate carries `selected_by = "overflow"`. Subtracting them gives the batch the preregistered mixture would have produced.

| id | provider  | reasoning type                                 | taken as | question                                                                                                                                                                                                                                        |
|----|-----------|------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01 | anthropic | <code>ambiguity_disambiguation</code>          | target   | In Web fetch tool, what does the <code>invalid_tool_input</code> field mean, and how does that differ from Tool search tool?                                                                                                                    |
| 02 | anthropic | <code>configuration_interaction</code>         | target   | What happens when <code>jwt.type</code> is <code>discovery</code> and no <code>discovery_base</code> is set?                                                                                                                                    |
| 03 | anthropic | <code>error_behavior</code>                    | target   | What happens when filtering by a non-existent <code>deployment_id</code> ?                                                                                                                                                                      |
| 04 | anthropic | <code>error_behavior</code>                    | target   | What happens if <code>encrypted_content</code> is missing or modified?                                                                                                                                                                          |
| 05 | anthropic | <code>exact_lookup</code>                      | target   | What must <code>allowed_domains</code> be?                                                                                                                                                                                                      |
| 06 | anthropic | <code>lifecycle_compatibility_migration</code> | target   | Where is <code>fallbacks</code> supported?                                                                                                                                                                                                      |
| 07 | anthropic | <code>lifecycle_compatibility_migration</code> | target   | Is <code>compaction_control</code> still supported?                                                                                                                                                                                             |
| 08 | anthropic | <code>lifecycle_compatibility_migration</code> | target   | Where is <code>budget_tokens</code> supported?                                                                                                                                                                                                  |
| 09 | openai    | <code>configuration_interaction</code>         | target   | What happens if the arguments are malformed JSON, are valid JSON but not an object (for example, <code>null</code> or a list), or contain non-standard constants such as <code>NaN</code> , <code>Infinity</code> , or <code>-Infinity</code> ? |
| 10 | openai    | <code>configuration_interaction</code>         | target   | What does <code>betas</code> override?                                                                                                                                                                                                          |

| id | provider | reasoning type                    | taken as | question                                                                                              |
|----|----------|-----------------------------------|----------|-------------------------------------------------------------------------------------------------------|
| 11 | openai   | configuration_interaction         | target   | What does <code>AWS_BEDROCK_BASE_URL</code> override?                                                 |
| 12 | openai   | configuration_interaction         | target   | What does <code>FuseMountPattern</code> require?                                                      |
| 13 | openai   | configuration_interaction         | target   | What does <code>S3FilesMountPattern</code> require?                                                   |
| 14 | openai   | configuration_interaction         | overflow | What does <code>ApplyPatchTool</code> require?                                                        |
| 15 | openai   | configuration_interaction         | overflow | What happens when a <code>[ ComputerTool ]</code> <code>[agents.tool.ComputerTool]</code> is present? |
| 16 | openai   | configuration_interaction         | overflow | What happens when you use a <code>Session</code> (e.g., <code>SQLiteSession</code> )?                 |
| 17 | openai   | error_behavior                    | target   | What happens if we exceed the <code>max_turns</code> passed?                                          |
| 18 | openai   | exact_lookup                      | target   | What must <code>dispatcher</code> be?                                                                 |
| 19 | openai   | lifecycle_compatibility_migration | target   | What happened to <code>httpAgent</code> ?                                                             |

## 4. Lane B — multi-hop, searched the other way round

Batch 004 tested every pair of facts sharing a plausible identifier: 559 pairs, one chain. This batch searches dependency-first — a chain may only open on a sentence that *states a dependency* and puts its entity into a state, and only then is a consumer sought.

| stage                       | pairs |
|-----------------------------|-------|
| dependency pairs considered | 3     |
| failed span independence    | 1     |
| failed semantic equivalence | 1     |
| passed                      | 1     |

Budget 1000 pairs; 3 were considered, so the budget was not the constraint. 122 entities appear in at least one sentence that states a dependency. **1** survived every gate — and it is the chain batch 004 already holds, so it is a duplicate and **0** reached the batch.

### WHAT THAT COMPARISON IS WORTH

Starting from sentences that state a dependency removed 99% of the search and found the same thing. It does not conjure chains that are not there: the corpus supports very few, and that remains the finding rather than a problem with the method.

## 5. The two checks batch 004 paid for

### Semantic bridge equivalence

A bridge entity must mean the same thing in both spans. The check reads whether an identifier is used as a request parameter, an enum value, a field name or a type, and refuses a pair whose roles differ.

```
span 1 – `budget_tokens` can exceed `max_tokens` here...
span 2 – The loop exits on any other stop reason (`"end_turn"`, `"max_tokens"`, ...)
```

`max_tokens` is a **request parameter** in span 1 and an **enum value** in span 2: the same string naming two different things. Batch 004's composer could not see this, and the pair reached its near-miss diagnostic. It is now a regression test.

### An interaction must name two settings

A single conditional fact is not an interaction. Batch 004 shipped two under that label and its review relabelled both; here the miner requires two named settings and a documented relation between them, and the self-review drops a candidate whose evidence does not put its subject on the right side of that relation.

## 6. The self-review, and what it says about the precheck

Every candidate below had already passed the structural precheck — hashes, offsets, string containment, anaphora, size caps. The self-review then dropped **27 of 46**.

| drop reason        | candidates |
|--------------------|------------|
| GENERIC_IDENTIFIER | 9          |
| CLAIM_SCOPE        | 8          |
| CATEGORY           | 7          |
| QUESTION_SCOPE     | 2          |
| QUESTION_FORM      | 1          |

That ratio is the point restated with a larger sample than batch 004 gave: `precheck_holdout_ready` means **structurally capable** — not semantically correct, not human-approved, not holdout-eligible. The self-review is itself only authoring: it is not independent verification, and it confers nothing.

### Removed before export

| reason                          | count |
|---------------------------------|-------|
| unbuildable                     | 1260  |
| dropped by semantic review      | 27    |
| duplicate evidence              | 26    |
| duplicate question              | 18    |
| short of target before overflow | 14    |
| duplicate evidence text         | 3     |
| reasoning type ceiling          | 3     |
| excluded known failure case     | 1     |
| blocking anaphora               | 1     |
| not selected diversity          | 0     |

## 7. Coverage, and the projection

Confirmed today: **67** eligible cases. If every one of the 19 batch-005 candidates were eventually approved, the project would hold **86**. No batch has approved everything — the four closed batches approved 16 of 18, 17 of 18, 20 of 20 and 14 of 15 — so treat it as a ceiling.

The projection must not become a reason to approve. A hundred cases assembled by relaxing the bar measures less than sixty-seven assembled without.

## 8. Invariants

- No retrieval system was run against any candidate. `retrieval_was_not_run = true` ; `systems_executed = []` .
- SYSTEM-A `9afcb5b7...` and SYSTEM-B `304c3509...` remain frozen and unexecuted.
- The holdout is not frozen ( `holdout_frozen = false` ).
- Batches 001–004 are byte-identical and their closure hashes still verify.
- No batch-005 candidate is verified, approved or eligible. The next step is independent review.

---

Generated by scripts/build\_batch\_005\_pdf.py from gold\_review\_batch\_005.json, GOLD-001-batch-005-generation-report.json and GOLD-001-eligibility-status.json. Every figure is read from those artifacts at build time, and the build refuses to run if the report disagrees with the batch, if any candidate claims verification, or if the holdout has been frozen. Batch 005, schema 1.3, corpus snapshot snap\_689e336380a054d8039dc35b2c09cd0a, batch\_sha256 37bf3509a9205637588730760dc14bf2..., git commit f4c7d4e11a97. Raw provider documentation is not redistributed; quoted spans are the short excerpts under review.